

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ**

ЗВІТ

**до лабораторної роботи №5
з дисципліни "Програмування мікроконтролерів"**

Студента 2-го курсу
спеціальності
123 «Комп'ютерна інженерія»
Савчака Дениса Андрійовича

м. Ужгород – 2026 р.

Лабораторна робота №5

Варіант 9

Тема: Динамічна індикація, таймери, робота з логічним аналізатором у інтегрованому середовищі PSoC® Creator™

Мета: Ознайомитися з принципами роботи динамічної індикації та налаштуванні переривань у мікроконтролерних системах на базі PSoC. Набути практичних навичок створення проєктів у інтегрованому середовищі PSoC® Creator™, налаштування апаратних компонентів для керування індикаторами через зсувні регістри та реалізації ефективного виведення даних. Також набути практичних навичок використання логічного аналізатора на прикладі Saleae Logic.

Хід роботи

Завдання 1. Реалізувати циклічне виведення цифр від 7 до 0 на семисегментних індикаторах

Проект:

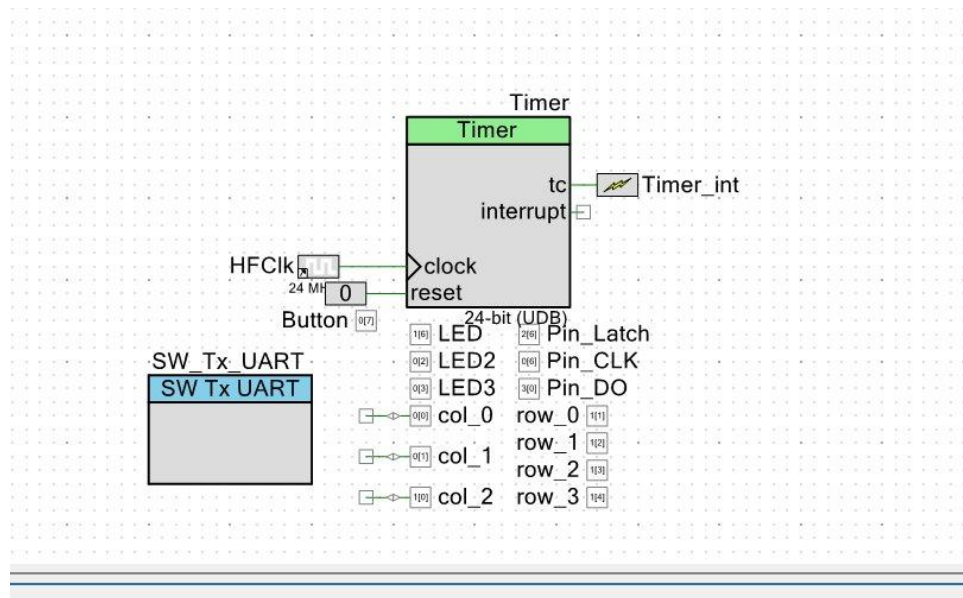


Рисунок 1 — Схема проекта TopDesign.cysch

Код:

```
#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90
};

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++) {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position,
                                           uint8_t segment_code)
{
    FourDigit74HC595_sendData(0xFF & ~(1 << position));
    FourDigit74HC595_sendData(segment_code);
    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

static volatile uint8_t led_counter = 0;

CY_ISR(Timer_Int_Handler2)
{

```

```

const uint8_t data[8] = {7, 6, 5, 4, 3, 2, 1, 0};
FourDigit74HC595_sendOneDigit(led_counter,
                               LED_NUM[data[led_counter]]);
led_counter++;
if (led_counter > 7) led_counter = 0;
}

int main(void)
{
    CyGlobalIntEnable;
    Timer_int_StartEx(Timer_Int_Handler2);
    Timer_Start();
    for (;;) {}
}

```

При запуску програми на семисегментних індикаторах відображається послідовність цифр від 7 до 0. Динамічна індикація реалізована через обробник переривань таймера, який кожні 1 мс оновлює один розряд дисплею.

Завдання 2. Вивести на дисплей число та декодувати його за допомогою логічного аналізатора.

Число = 11 (номер по списку) + 24 = 35.

Код:

```

#include "project.h"

#define MY_NUMBER    35u

static const uint8_t SEG[10] = {
    0xC0u, 0xF9u, 0xA4u, 0xB0u, 0x99u,
    0x92u, 0x82u, 0xF8u, 0x80u, 0x90u
};

static uint8_t disp_buf[8];
static uint8_t disp_pos;

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++) {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
    }
}

```

```

        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position,
                                           uint8_t segment_code)
{
    FourDigit74HC595_sendData(0xFF & ~(1 << position));
    FourDigit74HC595_sendData(segment_code);
    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

CY_ISR(Timer_Int_Handler2)
{
    FourDigit74HC595_sendOneDigit(dispos, disp_buf[dispos]);
    disp_pos = (disp_pos + 1u) & 7u;
}

int main(void)
{
    CyGlobalIntEnable;
    Timer_int_StartEx(Timer_Int_Handler2);
    Timer_Start();
    uint32_t num = MY_NUMBER;
    for (int8_t i = 7; i >= 0; i--) {
        disp_buf[i] = SEG[num % 10u];
        num /= 10u;
    }
    for (;;) {}
}

```

Результат (Logic analyzer):



Рисунок 2 — Декодування SPI-сигналу для числа 35

На рисунку 2 зображено декодований SPI-сигнал. Значення 0xB0 відповідає сегментному коду цифри 3, а 0x92 — цифри 5. Значення 0xC0 відповідає цифрі 0 на решті розрядів. Частота оновлення ~1.028 кГц відповідає таймеру з періодом 1 мс.

Завдання 3. Додати можливість вводу паролю на семисегментний дисплей. Перевірити введений пароль з тим, який оголошено в коді. Якщо пароль вірний — виконати певну дію; якщо невірний — сповістити користувача іншим паттерном.

Код:

```
#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90
};

#define SEG_OFF 0xFF
#define SEG_S 0x92
#define SEG_U 0xC1
#define SEG_C 0xC6
#define SEG_E 0x86
#define SEG_d 0xA1
#define SEG_n 0xAB
#define SEG_I 0xF9

#define STATE_INPUT 0
#define STATE_SUCCESS 1
#define STATE_DENIED 2
#define STATE_REINPUT 3

#define LED_GREEN_Write LED2_Write
#define LED_RED_Write LED3_Write
#define LED_ON 0u
#define LED_OFF 1u

static const uint8_t PASSWORD[4] = {1, 2, 3, 4};
static const uint8_t SUCCESS_MSG[7] = {
    SEG_S, SEG_U, SEG_C, SEG_C, SEG_E, SEG_S, SEG_S
};
static const uint8_t DENIED_MSG[6] = {
    SEG_d, SEG_E, SEG_n, SEG_I, SEG_E, SEG_d
};

static volatile uint8_t disp_buf[8];
static volatile uint8_t led_counter = 0;
static volatile uint8_t timer_minutes = 2;
static volatile uint8_t timer_seconds = 0;
static volatile uint32_t timer_ms = 0;
static volatile uint8_t timer_active = 0;
```

```

static volatile uint32_t scroll_tick = 0;
static volatile uint8_t scroll_pos = 0;
static volatile uint32_t blink_tick = 0;
static volatile uint8_t blink_visible = 1;

static uint8_t state = STATE_INPUT;
static uint8_t entered[4];
static uint8_t input_pos = 0;
static uint8_t reinput_pos = 0;

static void (*COLUMN_x_SetDriveMode[3])(uint8_t) = {
    col_0_SetDriveMode, col_1_SetDriveMode, col_2_SetDriveMode
};
static void (*COLUMN_x_Write[3])(uint8_t) = {
    col_0_Write, col_1_Write, col_2_Write
};
static uint8 (*ROW_x_Read[4])(void) = {
    row_0_Read, row_1_Read, row_2_Read, row_3_Read
};
static const int8_t KEYMAP[4][3] = {
    {1,2,3},{4,5,6},{7,8,9},{-1,0,-2}
};

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++) {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1); Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t pos, uint8_t
seg)
{
    FourDigit74HC595_sendData(0xFF & ~(1 << pos));
    FourDigit74HC595_sendData(seg);
    Pin_Latch_Write(1); Pin_Latch_Write(0);
}

static void clearAll(void) {
    for (uint8_t i=0;i<8;i++) disp_buf[i]=SEG_OFF;
}

static void updateTimer(void) {
    disp_buf[0]=LED_NUM[timer_minutes/10];
    disp_buf[1]=LED_NUM[timer_minutes%10]&0x7F;
    disp_buf[2]=LED_NUM[timer_seconds/10];
    disp_buf[3]=LED_NUM[timer_seconds%10];
}

static void updateSuccessScroll(void) {
    if (blink_visible)
        for (uint8_t i=0;i<8;i++)
            disp_buf[i]=SUCCESS_MSG[(scroll_pos+i)%7];
    else

```

```

        for (uint8_t i=0;i<8;i++) disp_buf[i]=SEG_OFF;
    }
    static void updateDeniedScroll(void) {
        if (blink_visible)
            for (uint8_t i=0;i<8;i++)
                disp_buf[i]=DENIED_MSG[(scroll_pos+i)%6];
        else
            for (uint8_t i=0;i<8;i++) disp_buf[i]=SEG_OFF;
    }

    CY_ISR(Timer_Int_Handler2)
    {
        if (timer_active) {
            timer_ms++;
            if (timer_ms >= 1000) {
                timer_ms = 0;
                if (timer_minutes==0 && timer_seconds==0)
                    timer_active=0;
                else {
                    if (timer_seconds==0){timer_minutes--
;timer_seconds=59;}
                    else timer_seconds--;
                }
                if (state==STATE_REINPUT) updateTimer();
            }
        }
        scroll_tick++;
        if (scroll_tick >= 400) {
            scroll_tick = 0; scroll_pos++;
            if (state==STATE_SUCCESS && scroll_pos>=7) scroll_pos=0;
            else if (state==STATE_DENIED && scroll_pos>=6)
                scroll_pos=0;
        }
        blink_tick++;
        if (blink_tick >= 500) {
            blink_tick=0; blink_visible=!blink_visible;
            if (state==STATE_SUCCESS) {
                updateSuccessScroll();
                LED_GREEN_Write(blink_visible?LED_ON:LED_OFF);
            } else if (state==STATE_DENIED) {
                updateDeniedScroll();
                LED_RED_Write(blink_visible?LED_ON:LED_OFF);
            }
        }
    }

    FourDigit74HC595_sendOneDigit(led_counter,disp_buf[led_counter]);
    if (++led_counter>=8) led_counter=0;
}

static int8_t scanKeypad(void)
{
    for (int c=0;c<3;c++) {
        COLUMN_x_SetDriveMode[c](col_0_DM_STRONG);
    }
}

```

```

        COLUMN_x_Write[c](0);
        for (int r=0;r<4;r++) {
            if (ROW_x_Read[r]()==0) {
                COLUMN_x_SetDriveMode[c](col_0_DM_DIG_HIZ);
                return KEYMAP[r][c];
            }
        }
        COLUMN_x_SetDriveMode[c](col_0_DM_DIG_HIZ);
    }
    return -3;
}

static uint8_t checkPassword(void) {
    for (int i=0;i<4;i++) if (entered[i]!=PASSWORD[i]) return 0;
    return 1;
}

static void ledsOff(void) {
    LED_GREEN_Write(LED_OFF); LED_RED_Write(LED_OFF);
}

static void goSuccess(void) {
    state=STATE_SUCCESS; timer_active=0;
    scroll_pos=0; scroll_tick=0; blink_tick=0; blink_visible=1;
    ledsOff(); LED_GREEN_Write(LED_ON); updateSuccessScroll();
}

static void goDenied(void) {
    state=STATE_DENIED;
    scroll_pos=0; scroll_tick=0; blink_tick=0; blink_visible=1;
    ledsOff(); LED_RED_Write(LED_ON); updateDeniedScroll();
}

int main(void)
{
    CyGlobalIntEnable;
    for (int c=0;c<3;c++)
        COLUMN_x_SetDriveMode[c](col_0_DM_DIG_HIZ);
    ledsOff(); clearAll();
    Timer_int_StartEx(Timer_Int_Handler2);
    Timer_Start();
    int8_t last_key=-3;
    for (;;) {
        int8_t key=scanKeypad();
        if (key!=last_key && key>=0 && key<=9) {
            if (state==STATE_INPUT) {
                if (input_pos<4) {
                    entered[input_pos]=key;
                    disp_buf[input_pos]=LED_NUM[key];
                    input_pos++;
                    if (input_pos==4) {
                        if (checkPassword()) goSuccess();
                        else {
                            timer_minutes=2;timer_seconds=0;
                            timer_ms=0;timer_active=1; goDenied();
                        }
                    }
                }
                input_pos=0;
            }
        }
    }
}

```

```

        }
    }
} else if (state==STATE_DENIED) {
    state=STATE_REINPUT; reinput_pos=0; ledsOff();
    updateTimer();
    disp_buf[4]=LED_NUM[key];
    disp_buf[5]=disp_buf[6]=disp_buf[7]=SEG_OFF;
    entered[0]=key; reinput_pos=1;
} else if (state==STATE_REINPUT) {
    if (reinput_pos<4) {
        entered[reinput_pos]=key;
        disp_buf[4+reinput_pos]=LED_NUM[key];
        reinput_pos++;
        if (reinput_pos==4) {
            if (checkPassword()) goSuccess();
            else goDenied();
            reinput_pos=0;
        }
    }
}
}
last_key=key; CyDelay(20);
}
}

```

При введенні чотирьох цифр пароля на індикаторах відображаються введені цифри. Після введення всіх 4 цифр система перевіряє пароль. При правильному паролі {1,2,3,4} на всіх 8 розрядах з'являється мигаючий напис «SUCCESS» і вмикається зелений LED. При неправильному — мигаючий напис «dEnIEd» і червоний LED. При невдалій спробі вводу запускається таймер блокування на 2 хвилини.

Завдання 4. Виконати циклічний вивід набору символів на семисегментних індикаторах. Варіант 9: Кожен другий на 2 позиції вправо.

Код:

```

#include "project.h"

static const uint8_t SEG[10] = {
    0xC0u, 0xF9u, 0xA4u, 0xB0u, 0x99u,
    0x92u, 0x82u, 0xF8u, 0x80u, 0x90u
}

```

```

};

static uint8_t disp_buf[8];
static uint8_t disp_pos;
static volatile uint32_t ms_cnt;

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++) {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1); Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t pos, uint8_t
seg)
{
    FourDigit74HC595_sendData(0xFF & ~(1 << pos));
    FourDigit74HC595_sendData(seg);
    Pin_Latch_Write(1); Pin_Latch_Write(0);
}

CY_ISR(Timer_Int_Handler2)
{
    FourDigit74HC595_sendOneDigit(disp_pos, disp_buf[disp_pos]);
    disp_pos = (disp_pos + 1u) & 7u;
    ms_cnt++;
}

static uint32_t get_ms(void) {
    uint32_t v;
    uint8_t s = CyEnterCriticalSection();
    v = ms_cnt; CyExitCriticalSection(s); return v;
}

int main(void)
{
    CyGlobalIntEnable;
    Timer_int_StartEx(Timer_Int_Handler2);
    Timer_Start();

    static const uint8_t seq[8] = {0,1,2,3,4,5,6,7};
    uint8_t even_off = 0, odd_off = 0;
    uint32_t last_tick = get_ms();

    for (;;) {
        uint32_t now = get_ms();
        if (now - last_tick >= 300u) {
            last_tick = now;
            even_off = (even_off + 1u) % 8u;
            odd_off = (odd_off + 2u) % 8u;
            disp_buf[0] = SEG[seq[even_off]];
            disp_buf[2] = SEG[seq[(even_off+2u)%8u]];
        }
    }
}

```

```

        disp_buf[4] = SEG[seq[(even_off+4u)%8u]];
        disp_buf[6] = SEG[seq[(even_off+6u)%8u]];
        disp_buf[1] = SEG[seq[(odd_off+1u)%8u]];
        disp_buf[3] = SEG[seq[(odd_off+3u)%8u]];
        disp_buf[5] = SEG[seq[(odd_off+5u)%8u]];
        disp_buf[7] = SEG[seq[(odd_off+7u)%8u]];
    }
}
}

```

Реалізовано циклічний зсув послідовності цифр 0–7 на 8 розрядах дисплею. Парні позиції (0, 2, 4, 6) зсуваються на 1 крок вправо кожні 300 мс, непарні позиції (1, 3, 5, 7) — на 2 кроки вправо кожні 300 мс, що відповідає варіанту 9 «Кожен другий на 2 позиції вправо».

Завдання 5. На перших чотирьох індикаторах реалізувати відлік 5 хвилин.

Код:

```

#include "project.h"

static const uint8_t SEG[10] = {
    0xC0u, 0xF9u, 0xA4u, 0xB0u, 0x99u,
    0x92u, 0x82u, 0xF8u, 0x80u, 0x90u
};

static uint8_t disp_buf[8];
static uint8_t disp_pos;
static volatile uint32_t ms_cnt;
static volatile uint8_t minutes = 5;
static volatile uint8_t seconds = 0;
static volatile uint32_t ms_counter = 0;

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++) {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1); Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t pos, uint8_t seg)

```

```

{
    FourDigit74HC595_sendData(0xFF & ~(1 << pos));
    FourDigit74HC595_sendData(seg);
    Pin_Latch_Write(1); Pin_Latch_Write(0);
}

static void updateDisplay(void)
{
    disp_buf[0] = SEG[minutes / 10];
    disp_buf[1] = SEG[minutes % 10] & 0x7F;
    disp_buf[2] = SEG[seconds / 10];
    disp_buf[3] = SEG[seconds % 10];
    disp_buf[4] = disp_buf[5] = disp_buf[6] = disp_buf[7] = 0xFF;
}

CY_ISR(Timer_Int_Handler2)
{
    ms_counter++;
    if (ms_counter >= 1000) {
        ms_counter = 0;
        if (!(minutes == 0 && seconds == 0)) {
            if (seconds == 0) { minutes--; seconds = 59; }
            else seconds--;
        }
        updateDisplay();
    }
    FourDigit74HC595_sendOneDigit(disp_pos, disp_buf[disp_pos]);
    if (++disp_pos >= 8) disp_pos = 0;
}

int main(void)
{
    CyGlobalIntEnable;
    updateDisplay();
    Timer_int_StartEx(Timer_Int_Handler2);
    Timer_Start();
    for (;;) {}
}

```

При запуску програми на перших чотирьох розрядах семисегментного дисплею відображається таймер зворотного відліку від 05:00 до 00:00. Другий розряд містить крапку, яка відокремлює хвилини від секунд. Таймер реалізовано через апаратне переривання з підрахунком мілісекунд. Позиції 4–7 залишаються порожніми.

Висновки

Під час виконання лабораторної роботи було засвоєно навички використання семисегментних індикаторів для виведення повідомлень, символів та таймера в інтегрованому середовищі PSoC® Creator™. Було використано програмне забезпечення Saleae Logic для дешифрування символів, що виводяться на індикатор. Реалізовано 5 завдань: циклічне виведення цифр від 7 до 0, виведення числа 35 з декодуванням SPI-сигналу, введення паролю з виведенням повідомлень SUCCESS/dEnIEd та керування кольоровими LED, циклічний зсув символів відповідно до варіанту 9, а також таймер зворотного відліку 5 хвилин.